Scripting Languages - Lecture 04

Lecture overview

Lecture 04 establishes core Python building blocks by introducing custom functions, dictionaries, and tuples as essential tools for modularity and data management. Functions are defined using the `def` keyword to encapsulate reusable logic, allowing complex code to be written once and invoked repeatedly without understanding internal details. While defining a function stores it in memory, execution requires a specific call that matches arguments to parameters; these functions can accept inputs, perform operations, and use the `return` statement to pass results back to the caller, whereas omitting `return` implicitly yields `None`. The lecture emphasizes that `return` immediately halts function execution, enabling conditional logic to exit early, and distinguishes between printing output to the screen versus returning values for further variable assignment and processing.

The session transitions to data structures, contrasting lists with dictionaries and tuples to highlight their unique properties. Dictionaries serve as unordered collections of key-value pairs that enable efficient data retrieval via hashing, unlike lists which rely on sequential position-based access. This structure is particularly effective for tasks like counting word frequencies in text analysis, where keys represent items (such as words or characters) and values act as incrementing counters. Tuples are presented as immutable sequences similar to lists but distinguished by their inability to be modified after creation; this immutability makes tuples suitable for use as dictionary keys, a role mutable lists cannot fulfill. The lecture concludes by summarizing the hierarchy of these data types and assigning practical exercises involving code-along notebooks and textbook readings on functions and dictionaries.

Slide 1: faculty of arts

This lecture introduces essential Python building blocks: constructing custom functions beyond built-in ones, utilizing dictionaries as versatile data structures for retrieving information, and exploring tuples as list-like sequences. The session progresses from defining these concepts to demonstrating their practical application within the scripting language context.

Slide 2: Functions

In programming, a function is a sequence of statements executed together via a single call, analogous to the mathematical mapping of an input to an output. Functions enable modularity by allowing complex code to be written once and reused repeatedly, avoiding superfluous work when executing identical logic multiple times. They serve as understandable building blocks for larger programs and encapsulate complexity, meaning users can leverage a function's capabilities without needing to understand its internal implementation details.

Slide 3: Functions

Functions are predefined blocks of code, such as `print`, `len`, and `type`, that can be invoked via a function call using their name followed by parentheses containing optional arguments. While functions typically accept input to produce a return value, neither inputs nor outputs are strictly required. Beyond built-in options, programmers can define their own functions to encapsulate logic, ensuring that code changes need only be made in one definition rather than repeated throughout the script.

Slide 4: Defining functions

Functions in Python are defined using the `def` keyword followed by the function name, parentheses for arguments, and a colon, with the indented code block forming the function body. Executing a `def` statement merely stores the function's code and name in memory without running it; actual execution requires a separate function call. The provided examples, `print_refrain` and `print_stanza`, illustrate functions that take no arguments and produce no return values, relying instead on internal print statements to display output.

Slide 5: Calling functions

To execute a function, it must be invoked by its specific name followed by parentheses. The slide demonstrates this with `print_refrain()`, which outputs the lyrics "Twinkle, twinkle, little star," and `print_stanza()`, which displays "Up above the world so high." These examples illustrate that calling the function triggers the code block associated with that name to run and produce its defined output.

Slide 6: Re-using functions

Functions enable code reuse by allowing previously defined building blocks, such as `print_refrain` and `print_stanza`, to be called within new functions like `print_full_lyrics`. This approach structures code efficiently by defining logic once rather than repeating print statements, thereby eliminating superfluous work. While the current example is simple, this principle establishes a foundation for creating specialized functions that process inputs and generate outputs effectively.

Slide 7: Function arguments

Some functions require inputs known as arguments to drive code execution. Within the function definition, these inputs are assigned to variables called parameters, which represent the values the function expects. Conversely, an argument is the specific value provided when calling the function, with one or multiple arguments defined within the parentheses following the function name.

Slide 8: Function arguments

Function arguments provide data to functions, where specific values like 'David' are assigned to parameters such as 'name' upon calling. The `display_greeting` example demonstrates how the function's output dynamically changes based on this input argument. Crucially, arguments must be supplied in a specific order and match the exact number of parameters defined in the function to execute correctly.

Slide 9: Returning results

Most functions compute a result from an input, requiring a `return` statement to make that value available for further processing or assignment to variables. Without this statement, variables defined inside the function possess local scope and remain inaccessible outside it, distinguishing returned values from simple screen prints. While functions do not strictly require inputs or outputs like mathematical functions, using `return` is essential for reusing computed results in subsequent code.

Slide 10: Returning results

When a function lacks an explicit `return` statement, it implicitly returns `None`. Additionally, placing a `return` statement anywhere within the function body immediately halts execution and exits the function.

Slide 11: Returning results

If a function lacks an explicit `return` statement, it implicitly returns `None`, meaning any output from internal `print` statements appears on screen but cannot be captured in a variable for further use. Conversely, using `return` sends the computed value back to the caller without printing it, allowing that result to be assigned and utilized later in the code. Additionally, placing a `return` statement anywhere within a function immediately halts its execution.

Slide 12: Returning results

If a function lacks an explicit `return` statement, it implicitly returns the special value `None`, representing "nothing." This occurs even if the function executes other operations like printing; for instance, assigning the result of a print-only function to a variable yields `None`. Additionally, placing a `return` statement anywhere within the function immediately halts its execution.

Slide 13: Returning results

If a function lacks an explicit `return` statement, it implicitly returns `None`, whereas including `return` immediately halts execution and sends a value back to the caller. Although return statements often appear at the end, they can be placed anywhere within the function definition, such as inside conditional branches to handle different cases like determining if a number is even or odd. This flexibility allows functions to exit early based on specific logic rather than always computing a single final value.

Slide 14: Returning results

Functions without an explicit `return` statement implicitly return `None`, as demonstrated when a function only prints output. The `return` statement can appear anywhere within the function body, and its execution immediately terminates the function while passing the specified value back to the caller. This mechanism allows functions like `even_odd` to conditionally return distinct strings such as 'even' or 'odd' based on input logic.

Slide 15: Functions: summary

Functions break programs into reusable, concentrated blocks of code that perform specific tasks well. In Python, a function is defined using the `def` keyword along with a name, parameters, and a code block; however, defining a function does not execute it. When called, arguments are matched to these parameters, allowing the function to process inputs and optionally return a result to the caller via the `return` statement.

Slide 16: Dictionaries

While lists organize data by natural sequential order for position-based access, dictionaries serve as unordered collections of key-value pairs designed for efficient data retrieval without scanning every item. Unlike lists where items are appended sequentially, dictionaries allow direct querying using specific keys, such as finding a word in a dictionary or selecting books by author. This structure is particularly valuable for efficiently organizing large datasets and performing tasks like counting word frequencies in text analysis.

Slide 17: Dictionaries

A Python dictionary stores unique key-value pairs using curly brackets and colons, enabling rapid value retrieval via keys regardless of dataset size. This efficiency stems from an underlying hash table implementation, which allows immediate access similar to locating a word in a physical dictionary by its starting letter. While dictionaries are unordered collections where duplicate keys overwrite existing entries, they serve as a fundamental building block for efficiently managing large amounts of data.

Slide 18: Properties of dictionaries

A dictionary maps immutable keys, such as numbers or strings, to values that can be any object type, including mutable lists. This structure enables very fast lookups while enforcing unique keys; if a key is reused, the new entry overwrites the previous value associated with it.

Slide 19: Working with dictionaries

Working with dictionaries involves accessing values via keys, where attempting to retrieve a non-existent key raises a `KeyError`, while the `in` operator safely tests for key existence. Dictionaries support dynamic modification through assignment and deletion, with `len()` returning the total number of entries. Additionally, the `.keys()`, `.values()`, and `.items()` methods provide access to the dictionary's keys, values, and (key, value) pairs respectively.

Slide 20: Lists vs. dictionaries: practical example

This slide illustrates a practical example of organizing a book collection using a Python list, where each element combines the author and title as a single string. The provided code defines a `books` list containing four entries: "Tolstoy - War and Peace", "Tolstoy - Anna Karenina", "Dostoevsky - Crime and Punishment", and "Dostoevsky - Brothers Karamazov". This structure serves as the baseline for comparing how lists handle data versus dictionaries, highlighting that lists store items sequentially without explicit key-value separation.

Slide 21: Lists vs. dictionaries: practical example

This slide contrasts organizing a book collection using a list versus a dictionary. While a list stores full titles as individual strings, a dictionary groups titles by author keys, mapping each author to a list of their works. This structure significantly simplifies retrieving all books by a specific author, such as accessing `books['Tolstoy']`, compared to searching through a flat list.

Slide 22: Using a dictionary to count

To count letter frequencies in a string, creating 26 individual variables is inefficient due to excessive conditional logic, while using a list requires complex character-to-index conversion. The most efficient approach utilizes a dictionary where characters serve as keys and their corresponding values act as counters that increment during iteration. This method avoids the structural limitations of the other options and allows for straightforward access to frequency data by iterating over the dictionary keys.

Slide 23: Using a dictionary to count

To count character occurrences in a string like 'interdisciplinary', initialize an empty dictionary and iterate through each character. If a character is not yet a key, add it with a value of 1; otherwise, increment its existing count by one. This approach efficiently builds a mapping of characters to their frequencies, which can later be filtered using `.items()` to display only entries meeting specific criteria, such as counts greater than or equal to four.

Slide 24: Counting words

Slide 24 introduces the task of counting words using a specific Python string variable containing an excerpt from Vladimir Nabokov's The Gift. The example text is a complex, multi-clause sentence describing a visual scene involving light, mirrors, and moving boughs, serving as the dataset for demonstrating word-counting operations in scripting languages. No additional procedural steps or algorithmic details are provided in the source material for this specific slide.

Slide 25: Counting words

To count words, the code first normalizes the input by applying `lower()` to handle case sensitivity and then uses `split()` with default space delimiters to convert the sentence into a list of 96 words. A dictionary named `word_counter` is initialized to store frequencies, iterating through the word list to either set a new entry to 1 or increment the existing count for each encountered word.

Slide 26: Looping over dictionaries

When looping over a dictionary in Python, the iteration process returns only the keys by default. To access the corresponding values during this loop, you must use each key to index into the dictionary explicitly, as demonstrated by iterating through `data` and printing both the key and `data[key]`.

Slide 27: Looping over dictionaries

Looping directly over a dictionary yields only its keys, requiring manual lookup to access corresponding values. Since accessing both keys and values is common, Python provides the `.items()` method to iterate through key-value pairs simultaneously. This approach allows unpacking both elements directly in the loop header, such as `for word, count in word_counter.items()`, eliminating the need for separate value retrieval.

Slide 28: How to print out words (and their frequencies) in alphabetical

To print words and their frequencies in alphabetical order, use the sorted list rather than the original word list. The sorted list contains the words arranged alphabetically, such as starting with "a" followed by "cross," while maintaining the correct frequency count for each entry. This approach ensures the output is properly ordered without needing to re-sort the data during printing.

Slide 29: How to print words with frequency of at least 4?

To print only words with a frequency of at least 4, iterate through the dictionary and check if each word's count is greater than or equal to 4. If this condition is met, print the corresponding key-value pair; otherwise, skip it. This process effectively filters the data to display only the four words in the example that meet the minimum frequency threshold, demonstrating how to isolate specific high-frequency entries from a larger dataset.

Slide 30: Tuples

Tuples are Python objects similar to lists but distinguished by their immutability; once created using parentheses, their elements cannot be changed. Because dictionary keys must be immutable, mutable lists cannot serve this purpose, whereas tuples can effectively act as keys. This is demonstrated by a `years_married` dictionary where tuple pairs like `('John', 'Mary')` function as valid keys mapping to integer values, a capability that would fail with lists due to their mutability.

Slide 31: Tuples

Tuples are immutable sequences where indexing works like lists, but item assignment raises a `TypeError` because elements cannot be changed after creation. They support unpacking to assign items directly to separate variables in a single statement. When comparing tuples, Python evaluates them item by item: the first element determines the result, while subsequent elements break ties only if the preceding ones are equal.

Slide 32: Sort words from longest to shortest

This slide introduces the specific task of sorting a list of words in descending order based on their character length. As part of the scripting languages module, this concept demonstrates how to manipulate string data by prioritizing longer entries over shorter ones. No additional examples or implementation details were provided in the aligned lecture material for this specific section.

Slide 33: Get most common words in a text

This section introduces the task of identifying the most frequent words within a given text, a fundamental operation in scripting languages for text analysis. As part of a broader series on scripting techniques, this concept serves as a practical example of how scripts can process and summarize textual data efficiently.

Slide 34: Summary of data types

This slide summarizes the hierarchy of Python data types, categorizing them into Objects, Iterables (including Sequences like mutable lists and immutable tuples/strings), Mappings (specifically dictionaries), and Numbers (floats and integers). It visually organizes these fundamental scripting language structures to clarify their relationships and mutability properties.

Slide 35: Practical session / reading

Practical session 3 requires students to access the lecture's code-along notebook on Toledo and re-run the examples at home. Additionally, students must read chapters 7 and 8 of the textbook Python for Everybody to complete the assigned reading.

Key points to remember

The session transitions to data structures, contrasting lists with dictionaries and tuples to highlight their unique properties. Dictionaries serve as unordered collections of key-value pairs that enable efficient data retrieval via hashing, unlike lists which rely on sequential position-based access. This structure is particularly effective for tasks like counting word frequencies in text analysis, where keys represent items (such as words or characters) and values act as incrementing counters. Tuples are presented as immutable sequences similar to lists but distinguished by their inability to be modified after creation; this immutability makes tuples suitable for use as dictionary keys, a role mutable lists cannot fulfill. The lecture concludes by summarizing the hierarchy of these data types and assigning practical exercises involving code-along notebooks and textbook readings on functions and dictionaries.